

Spring/SpringBoot

Spring: framework modular para Java (completo, mucha configuración).

Framework web: ayuda a crear, desarrollar y publicar aplicaciones como: Servicios web, APIs REST y recursos dinámicos (HTML, JSON, XML)

URL Routing: proceso de decirle a tu aplicación qué hacer cuando alguien entra a una dirección web (URL).

En Spring Boot, se hace con anotaciones como `@GetMapping`, `@PostMapping`, etc.

```
@RestController
public class HelloController {
    @GetMapping("/hello")
    public String hello() {
        return "Hola Erika!";
    }
}
```

Input Form Managing & Validation: Manejar formularios de usuarios (ejemplo: registro, login) y validar que la info sea correcta antes de guardarla.

Spring Boot usa anotaciones de **Java Bean Validation**

Bean: Un objeto gestionado por el **contenedor de Spring**.

En vez de crear objetos con `new`, Spring los administra, inicializa e inyecta donde se necesiten.

Se configuran con anotaciones:

`@Component` (clase genérica).

`@Service` (lógica de negocio).

`@Repository` (acceso a datos).

`@Bean` (cuando registras manualmente un objeto).

HTML, XML, JSON & Templating Engine: Formatos de datos o vista se puede procesar o devolver:

HTML → páginas web.

XML → formato estructurado (antiguo, aún usado).

JSON → el más común para APIs REST.

Templating engine: herramientas como **Thymeleaf** permiten crear páginas HTML dinámicas con datos del backend.

Database Connection & ORM

ORM (Object Relational Mapping): convierte tablas de una base de datos ↔ objetos Java.

El más usado en Spring es **Hibernate** (integrado en Spring Data JPA).

Web Security - JWT

JWT (JSON Web Token): token en formato JSON que permite autenticar usuarios sin guardar sesiones en el servidor.

Spring/SpringBoot

Session Repository & Retrieval: Manejo de sesiones de usuario (datos temporales de un usuario conectado).

Opciones:

- Guardar en memoria del servidor.
- Guardar en base de datos.
- Guardar en Redis (rápido y distribuido).

Spring Session facilita guardar/recuperar sesiones.

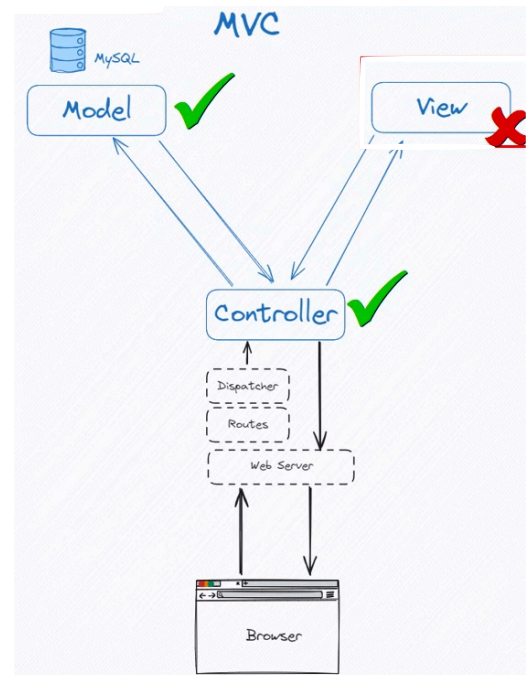
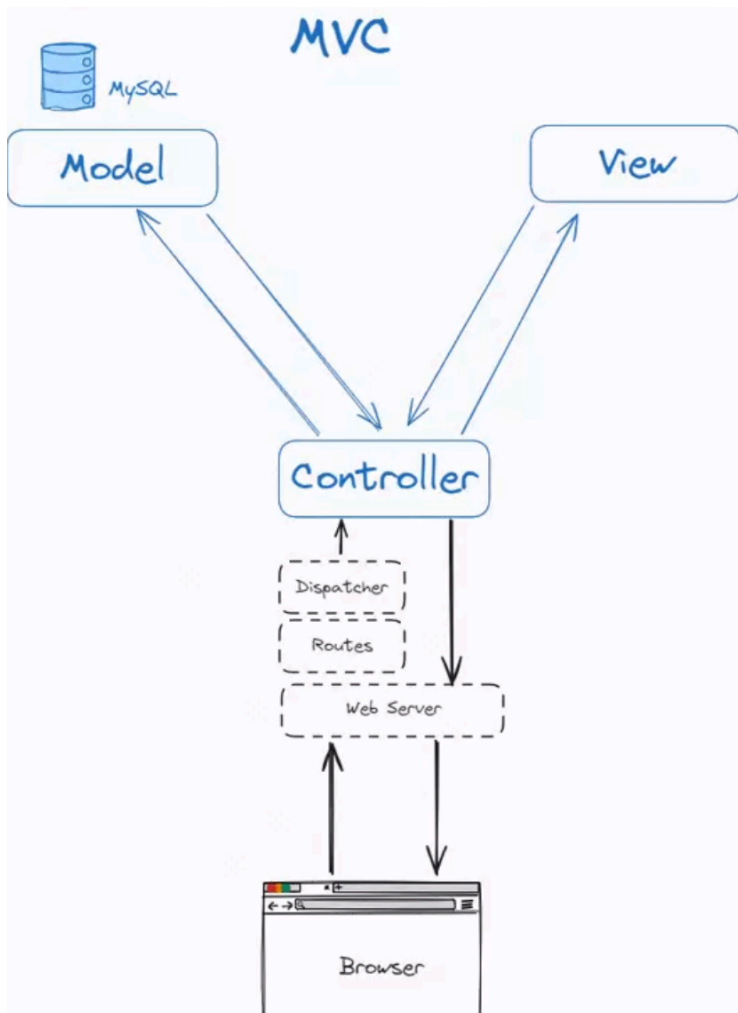
Spring/SpringBoot

MVC Architecture: Patrón Modelo-Vista-Controlador.

Modelo: la lógica/datos (clases `User`, repositorios JPA).

Vista: lo que ve el usuario (HTML, JSON).

Controlador: recibe la petición y decide qué responder.



Model : representa los datos y la lógica (ej. clases con JPA conectadas a MySQL).

Controller : recibe las peticiones, habla con el modelo y devuelve la respuesta.

View  (tachado): no se usa si trabajas con front-end separado (ejemplo: React, Angular, Vue).

En arquitecturas modernas (**SPA** o frontend separado):

El backend con Spring Boot ya no genera vistas, solo entrega JSON.

El frontend (ej. React, Angular) es quien construye las vistas.

Flujo sin Vista en el backend

El navegador pide datos al backend: GET `/api/users`.

El Controller busca en el Model (ej. `UserRepository`).

El Controller devuelve JSON como respuesta, no HTML.

El frontend (React, Angular, etc.) recibe ese JSON y lo dibuja en pantalla.

Si tu backend incluye vistas (Thymeleaf, JSP) → el MVC completo aplica.

Si tu backend es solo API REST → la "V" queda fuera, el frontend hace esa parte.

El patrón se entiende entonces más como **MC (Modelo + Controlador)** dentro del backend.

Spring/SpringBoot

Herramientas de construcción y autorización

- Apache Ant: herramienta antigua para automatizar compilación.
- Maven: más popular, basado en pom.xml (Project Object Model).
- Gradle: moderno, rápido, usa sintaxis Groovy/Kotlin.
- Apache Tomcat: servidor web Java de código abierto (Spring Boot lo integra embebido).
- **Repositorio de dependencias: Maven Repository.**
Dependencias comunes
 - spring-boot-starter-web: crear APIs REST.
 - spring-boot-starter-data-jpa: conectar con bases de datos.
 - spring-boot-starter-security: seguridad y autenticación.
 - spring-boot-starter-test: pruebas unitarias.

Spring Boot: Framework basado en Spring que simplifica el desarrollo de aplicaciones Java. Crea endpoint REST en minutos.

Reduce configuración manual, gracias a:

- **Autoconfiguración:** detecta automáticamente qué dependencias se tiene y configura beans.
- **Opinionated defaults:** ofrece configuraciones por defecto para que el proyecto funcione rápido.
- **Starter POMs:** dependencias listas para cada necesidad
- **Embedded server:** incluye Tomcat/Jetty/Undertow sin necesidad de instalar servidores externos.
- **Spring Initializr:** Herramienta para generar proyectos listos: start.spring.io.
 - Lenguaje: Java
 - Build Tool: Maven/Gradle
 - Dependencias: Web, JPA, Security, etc.
 - Descarga un proyecto base con todo configurado.

Anotaciones básicas

@SpringBootApplication: marca la clase principal.

@RestController: combina @Controller y @ResponseBody.

@GetMapping, @PostMapping, etc.: manejan endpoints HTTP.

@Autowired: inyección de dependencias.

@Entity, @Table, @Id: para trabajar con JPA y bases de datos.

***packaging:** cómo se organiza y distribuye una aplicación Java para poder ejecutarla o desplegarla en un servidor.

En Spring Boot, los formatos más comunes son JAR y WAR.

1. Servidor Web: programa que recibe las peticiones HTTP y devuelve respuestas (HTML, JSON, etc.).

Spring Boot trae un servidor embebido (por defecto Tomcat).

Spring/SpringBoot

2. JAR (Java ARchive): archivo ejecutable que empaqueta: código compilado, dependencias, recursos.

En Spring Boot se llama fat jar o uber jar porque incluye todo lo necesario para correr la app.

Ejemplo: demo-0.0.1-SNAPSHOT.jar

3. POM (Project Object Model)

Archivo **pom.xml** que usa **Maven** para declarar dependencias (ej. Spring Web, JPA), configurar plugins (para empaquetar en JAR o WAR) y definir información del proyecto (nombre, versión, etc.). Maven lee este archivo y baja las librerías necesarias de [Maven Repository](#)

4. **WAR (Web ARchive) (opcional):** Alternativa al JAR.

Antes de Spring Boot era lo típico: generabas un **WAR** y lo desplegabas en un servidor Tomcat externo.

Hoy ya casi no se usa, porque **Spring Boot + JAR autoejecutable** simplificó todo.

Servidor web: recibe peticiones → en Spring Boot está embebido (Tomcat).

JAR: paquete ejecutable con tu app + dependencias + Tomcat.

POM: receta del proyecto (qué dependencias y cómo empaquetar).

WAR: empaquetado clásico para desplegar en servidores externos (menos usado con Spring Boot).

Analogía

POM = la receta.

Maven/Gradle = el chef que sigue la receta.

JAR/WAR = el platillo ya preparado y listo para servir.

Servidor web (Tomcat) = el plato donde lo sirves para que el cliente (navegador) lo consuma.

Spring/SpringBoot

Ejemplo de project maven > I text PDF

IDE eclipse > New > Maven Project

New Maven Project:

marcar create a simple project (skip archetype selection) o next> y configurar manual

Seleccionar workspace

Working set según la organización de proyectos

Next >

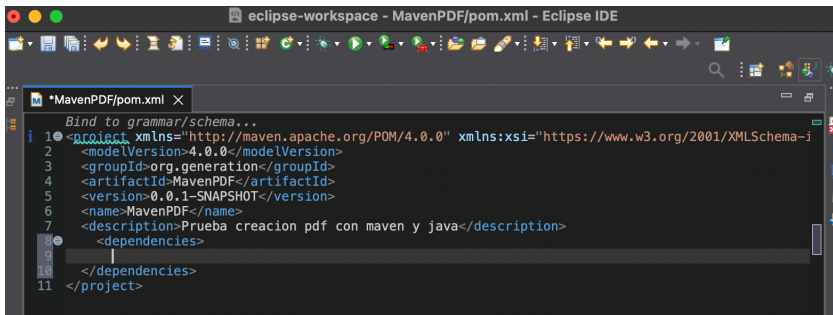
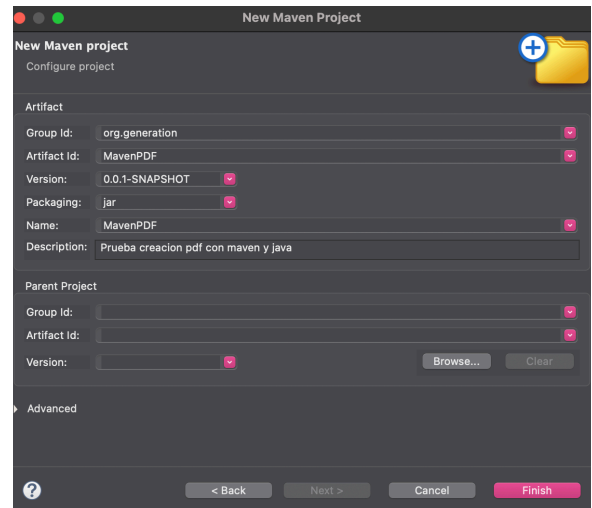
Group ID, artifact iD - nombrar

Descripción - si se desea agregar, también se puede editar en código una vez creado

Finish >

Maven/pom.xml:

Todas las etiquetas deben cerrar sin interferir en otras



Si marca error en la línea de project,

agregar S a xmlns:xsi

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="https://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

Etiquetas dependencies:

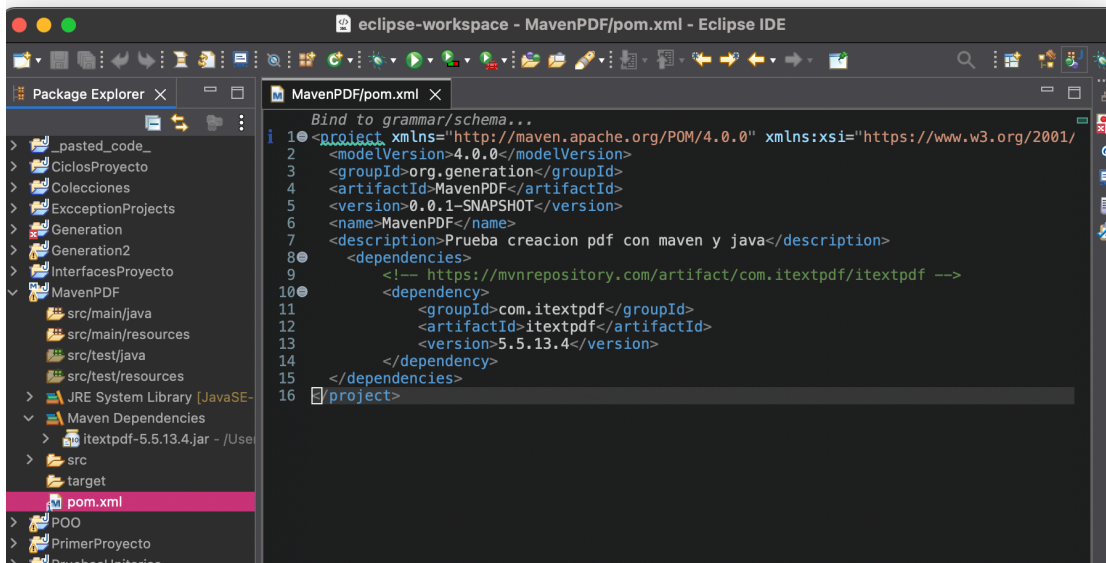
Dentro de esta pegar la dependencia que en este caso obtuvimos de mvn repository > elegir version > seleccionar maven o herramienta en uso > copiar compile



Spring/SpringBoot

*Clic derecho > Source > format

Guardar y automáticamente en el package explorer de la izquierda se vera reflejada la dependencia.



Crear archivo main y comenzar a trabajar en este caso un PDF, hacer uso de cntl espacio para importar los paquetes según requiera.

you employed

CH57 SA de CV de RL

recibo de nomina		
nombre		rfc:
fecha:	departamento:	salario:

